

Budget Distributed Support Vector Machine for Non-ID Federated Learning Scenarios

A. NAVIA-VÁZQUEZ*, Universidad Carlos III of Madrid
R. DÍAZ-MORALES and M. FERNÁNDEZ-DÍAZ, Tree Technology

In recent years, there has been remarkable growth in Federated Learning (FL) approaches because they have proven to be very effective in training large Machine Learning (ML) models and also serve to preserve data confidentiality, as recommended by the GDPR or other business confidentiality restrictions that may apply. Despite the success of FL, performance is greatly reduced when data is not distributed identically (non-ID) across participants, as local model updates tend to diverge from the optimal global solution and thus the model averaging procedure in the aggregator is less effective. Kernel methods such as Support Vector Machines (SVMs) have not seen an equivalent evolution in the area of privacy preserving edge computing because they suffer from inherent computational, privacy and scalability issues. Furthermore, non-linear SVMs do not naturally lead to federated schemes, since locally trained models cannot be passed to the aggregator because they reveal training data (they are built on Support Vectors), and the global model cannot be updated at every worker using gradient descent. In this article, we explore the use of a particular controlled complexity ("Budget") Distributed SVM (BDSVM) in the FL scenario with non-ID data, which is the least favorable situation, but very common in practice. The proposed BDSVM algorithm is as follows: model weights are broadcasted to workers, which locally update some kernel Gram matrices computed according to a common architectural base and send them back to the aggregator, which finally combines them, updates the global model, and repeats the procedure until a convergence criterion is met. Experimental results using synthetic 2D datasets show that the proposed method can obtain maximal margin decision boundaries even when the data is non-ID distributed. Further experiments using real-world datasets with non-ID data distribution show that the proposed algorithm provides better performance with less communication requirements than a comparable Multilayer Perceptron (MLP) trained using FedAvg. The advantage is more remarkable for a larger number of edge devices. We have also demonstrated the robustness of the proposed method against information leakage, membership inference attacks, and situations with dropout or straggler participants. Finally, in experiments run on separate processes/machines interconnected via the cloud messaging service developed in the context of the EU-H2020 MUSKETEER project, BDSVM is able to train better models than FedAvg in about half the time.

CCS Concepts: • **Theory of computation** → **Support vector machines**; *Distributed algorithms*; • **Computing methodologies** → **Supervised learning by classification**; • **Computer systems organization** → *Distributed architectures*; *Client-server architectures*; • **Software and its engineering** → *Cloud computing;*

This work has been partially supported by European Union's Horizon 2020 research and innovation programme under Grant Agreement No. 824988 ("MUSKETEER" project) and by FEDER/Ministerio de Ciencia, Innovación y Universidades - Agencia Estatal de Investigación under Grant No. TEC2017-83838-R.
Authors' addresses: A. Navia-Vázquez, Universidad Carlos III of Madrid Avda. Universidad, 30, 28911-Leganés, Madrid, Spain; email: angel.navia@uc3m.es; R. Díaz-Morales and M. Fernández-Díaz, Tree Technology, Parque Tecnológico de Asturias CL Faya 18 33428-Llanera, Spain; emails: roberto@treetk.com, marcos.fernandez@treelogic.com.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2022 Association for Computing Machinery.
2157-6904/2022/09-ART100 $15.00
https://doi.org/10.1145/3539734

Additional Key Words and Phrases: Kernel methods, budget, distributed, support vector machine, federated learning, non-ID, MUSKETEER, EU-H2020

ACM Reference format:
A. Navia-Vázquez, R. Díaz-Morales, and M. Fernández-Díaz. 2022. Budget Distributed Support Vector Machine for Non-ID Federated Learning Scenarios. *ACM Trans. Intell. Syst. Technol.* 13, 6, Article 100 (September 2022), 25 pages.
https://doi.org/10.1145/3539734

1 INTRODUCTION

The proliferation of machine learning services and some data leakage scandals that have hit the media lately are causing great concern in the population about the confidentiality of data, which has also motivated the appearance of data privacy regulations (e.g., the **EU General Data Protection Regulation** (**GDPR**) [6]). Very often data from different sources is collected in data centers, an approach that directly overrides confidentiality rights and also carries an additional potential risk of information leakage to third parties. To mitigate these concerns, data localization stems from regulations and administrative procedures such as GDPR, which restrict the free flow of data.

The field of **Privacy-Preserving Machine Learning** (**PPML**), which comprises **Secure Multiparty Computation** (**SMC**) approaches [7] or methods that are based on cryptography [52], aim to bridge the gap between the protection of personal data and the use of data. These approaches offer a high level of security under certain assumptions, but can generate huge additional computational overhead, often unaffordable, since they require not only supercomputing clusters but also the transmission of a large amount of data between users. We can also find other alternatives such as perturbation, averaging, and randomization based approaches (e.g., those induced by Differential Privacy principles [11, 46]), but they generally involve a compromise between accuracy and confidentiality because the datasets are altered in some way and can lose their usefulness for training predictive models.

A more recent approach, first introduced by Google [26], introduces a new training paradigm called **Federated Learning** (**FL**), which allows different devices to collaboratively learn a shared prediction model while keeping all the training data on premises, decoupling the ability to do machine learning from the need to store the data in a centralized environment [58]. The general procedure is to use the data from each participant to locally train the models at the edge nodes and send the updated models to a central counterparty or "aggregator," which averages them to obtain a global model. In recent years, several models that can be trained using gradient descent [26] (e.g., Multilayer Perceptrons, Convolutional Neural Networks) have received much attention due to their easy adaptation to Federated scenarios, being the averaging of federated models (FedAvg) the most popular approach [31].

Most machine learning models that work with distributed data assume that the data is **Independent and Identically Distributed** (**IID** assumption). However, in many practical cases, this assumption is not fulfilled, which poses serious convergence problems in some distributed scenarios (e.g., in FL). We will not make assumptions about the independence of the data and will concentrate on the "**non Identically Distributed**" case (hence, the "**non-ID**" term). More specifically, we will explore the common non-ID situation where participants have different proportions of labels (the extreme case is when every worker only has training patterns corresponding to one of the classes of interest). These non-ID situations in Federated settings are illustrated in works focused on heterogeneous label distribution over nodes [25, 28], those that address shifts in the feature space [29], clients with different degree of non-ID data [29], and other specific approaches for

multi-task learning [50, 63]. Approaches that share a global training dataset across all participants [24, 64] are contrary to FL principles and will not be considered here.

In addition to neural networks, other ML approaches such as Kernel Methods (e.g., **Support Vector Machines** (**SVMs**)) have been shown to provide very good and competitive solutions in centralized settings [44] for various problems, with the attractive feature of providing a minimum risk, maximum margin deterministic solution obtained by solving a centralized optimization problem. Despite this, their presence in FL scenarios is marginal, especially when it comes to kernelized (non-linear) models. SVMs suffer from inherent confidentiality issues, as the resulting models are built based on the support vectors, which are actually a subset of the training data and should not be exposed in any way, even during the training phase. SVMs also present scalability problems when the number of training patterns is high and also standard solvers (based on Quadratic Programming) are difficult to adapt to the FL setting.

Existing distributed SVM approaches mainly focus on computational, communication bandwidth or power saving perspectives, but rarely on confidentiality issues. It is widely recognized how challenging it is to distribute SVM computations across different nodes, especially in the non-linear case. That is, nonlinear SVM training requires solving a quadratic optimization problem using kernel calculations between any possible pair of training patterns, which is challenging when the training data is distributed among different participants. Also, local SVM training isn't much help either, since each resulting local model has a different architecture (different support vectors), and it's not possible to merge all those models in the aggregator to get a single global model. There are various approaches to parallel SVM computation where separate processes running on the same machine have access to shared memory segments (used for coordination of the different threads of execution). This is a different scenario than the distributed one considered here, where different processes are assumed to be on differently located machines and must be coordinated by exchanging messages over physical communication lines, and sometimes under "pervasive"[1] assumptions. Therefore, parallel SVM developments like [5, 62] are not relevant here, since they rely on shared memory instead of message passing. Most SVM solvers are iterative in nature, and very often the **support vectors** (**SVs**) must be communicated between participants, breaking data confidentiality. Methods based on incremental approaches [51], Iterative Exchange of Support Vectors [3, 13, 14], Cascade SVMs [17], or Projection methods as in [48] rely on this (non-confidential) model/SVs exchange mechanism. Other methods are designed for linear models only and also require the exchange of local models/solutions, such as the Alternating Direction Method of Multipliers [15, 16], Distributed Dual Ascend [20], Distributed Block Minimization [36], the resilient approach presented in [56, 57, 59], or the approaches described in [2, 4, 19, 21, 45, 55]. The Multi-Task Learning approach described in [50][2] was unsuccessful in our "extreme non-ID" benchmark scenario, as local optimizations cannot be resolved when there are only samples of one class. Some SVM approaches that claim to preserve data confidentiality are only available for vertical partitioning and require to exchange full kernel matrices, which are aggregated at the central node to solve the SVM optimization centrally, as in [30, 60, 61]. Some works propose to rebuild the complete Gram matrix in a central location, which raises serious scalability issues and also requires a direct communication between each possible pair of participants to compute all possible values in the Gram matrix. This fully connected architecture differs from the "star" architecture associated to FL [53, 60, 61]. In some cases, it is possible to apply SMC schemes but with

[1]Small devices communicating over unreliable networks with low bandwidth.

[2]The described multi-task approach (MOCHA), as the authors acknowledge, does not fit the standard assumptions of FL (sharing information between workers to obtain a centralized model at the aggregator), but instead aims to learn separate models in each node, using multitask principles to increase performance by sharing information between them.

the restriction of needing a set of non-collusive servers. Furthermore, the resulting models need the cooperation of all participants to provide new predictions using SMC protocols, since the final model contains SVs from all participants and these SVs are private and cannot be outsourced [60]. Some other approaches only provide means to classify new samples, but do not support model training [40]. Finally, some works present incremental versions of approximate solutions that are different from the SVM formulation, such as the Least Squares approach in [9].

With the work presented here, we intend to close the gap between a special type of SVMs (known as Budget SVMs (BSVMs)) and federated schemes, obtaining improved solutions in some applications, the proposed methods being especially robust in non-ID FL scenarios, as shown in the experimental section.

The structure of the article is as follows. In Section 2, we briefly review the main theoretical aspects of the SVM formulation, explaining in Section 2.1 the "budget" SVM approach. In Section 2.2, we present the FL implementation and in Section 2.3 we propose a method to reduce the amount of information to be transmitted in each round of communication. We also provide in Section 2.4 a preliminary risk analysis of the proposed scheme with respect to information leakage. In Section 3, we present the experimental results in the non-ID scenario, using both some synthetic problems for illustrative purposes (Section 3.1) and real-world datasets (Section 3.2). The performance of the algorithm in the presence of dropout events or straggler users is analyzed in Section 3.3 and the robustness of the proposed method w.r.t. membership inference attacks is analyzed in Section 3.4. We have also included results when the training takes place on a real Industrial Data Platform like MUSKETEER [49], with participants running on different machines and communicating via a cloud messaging service located in a different country (Section 3.5). Finally, in Section 4, we present the main conclusions and possible future research.

2 REVIEW OF SVMS

The SVM is a very powerful ML model that works very well in many practical scenarios and automatically obtains the model architecture (in terms of the so-called SVs) and the corresponding model weights. However, it suffers from two main drawbacks. The first is the excessive computational cost required to find the solution when the number of training patterns is very large (large **Quadratic Programming** (**QP**) optimization). The second is that the final model is built using a, potentially very large, collection of training patterns (the SVs), and therefore data confidentiality is lost if the trained model is shared. In an attempt to overcome these drawbacks, we have developed over the years efficient alternative solving methods based on Iterated Re-Weighted Least Squares [34, 39] and controlled complexity (semiparametric or "Budget") SVM models (BSVMs) [8, 33–35], as a means to control the size of the classifier by defining "*a priori*" the architecture of the model in the formulation of the SVM problem. In this section, we will briefly review those concepts and propose an SVM solution suitable for FL environments.

Consider a binary classification problem defined by a set of N_T labeled training patterns $\{\mathbf{x}_i, y_i\}_{i=1}^{N_T}$ with $\mathbf{x}_i \in \mathbb{R}^N$ and $y_i \in \{-1, +1\}$. The nonlinear or kernelized SVM first projects the input data into a high dimensional space F by means of a nonlinear projection $\boldsymbol{\phi}(\cdot)$ so that the inner products between projected vectors in F can be computed by means of a kernel function $k(\mathbf{x}_i, \mathbf{x}_j) = \langle \boldsymbol{\phi}(\mathbf{x}_i), \boldsymbol{\phi}(\mathbf{x}_j) \rangle = \boldsymbol{\phi}^T(\mathbf{x}_i)\boldsymbol{\phi}(\mathbf{x}_j)$. Then a maximum margin linear classifier is defined in F as $f(\mathbf{x}) = sign(\mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}) + b)$, such that the model parameters $\{\mathbf{w}, b\}$ are the solution to

$$\min_{\mathbf{w}, b, \xi_i} \left\{ \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{N_T} \xi_i \right\} \tag{1}$$

subject to

$$y_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b) - 1 + \xi_i \geq 0, \qquad \forall i = 1, \ldots, N_T, \tag{2}$$

$$\xi_i \geq 0, \qquad \forall i = 1, \ldots, N_T, \tag{3}$$

where ξ_i are positive slack variables introduced to deal with non-separable problems and C is the penalty for patterns incorrectly classified or inside the margin. After introducing the restrictions in the functional by means of Lagrange multipliers and applying the **Karush-Kuhn-Tucker** (**KKT**) optimality conditions, we obtain the optimal values $\{\mathbf{w}, b\}$, the solution to the problem given by (1), (2), and (3). Although F can be infinite dimensional (for instance, in the Gaussian kernel case), in which case $\mathbf{w}$ is also infinite dimensional, one of the KKT conditions tells us that $\mathbf{w}$ can actually be represented as a linear combination of projected datapoints $\boldsymbol{\phi}(\mathbf{x}_i)$ (i.e., $\mathbf{w}$ lies in the span of $\{\boldsymbol{\phi}(\mathbf{x}_i), i = 1, \ldots, N_T\}$) such that $\mathbf{w} = \sum_{i=1}^{N_T} y_i \alpha_i \boldsymbol{\phi}(\mathbf{x}_i)$. Also many dual parameters α_i usually become zero, further reducing the model complexity.

However, in this standard (dual space) SVM formulation, the number of SVs (patterns $\mathbf{x}_i$ with non-zero α_i) can be very high and we get very large models, expensive to deploy for massive prediction tasks. Also, as already discussed in Section 1, the SVs that define the model are actually training patterns, therefore we are potentially disclosing confidential information if we share the resulting model. In the next subsection, we will describe a BSVM solution that is free of these drawbacks.

2.1 The Budget SVM

To overcome the drawbacks mentioned above, we propose to assume a limited complexity approximation to $\mathbf{w}$ by using a set of P elements in F that can be obtained by projecting P pre-image[3] vectors $\mathbf{p} \in \mathbb{R}^N$, such that the "budget" hyperplane $\mathbf{w}_B$ is defined as:

$$\mathbf{w}_B = \sum_{i=1}^{P} \beta_i \boldsymbol{\phi}(\mathbf{p}_i) \tag{4}$$

and the optimization is solved to find the optimal β_i values. In previous works, we have explored different methods to select the best $\mathbf{p}_i$ values for a given problem such as random selection, greedy growth methods, or clustering [8, 34, 35]. We have observed that the final model performance is not very sensitive with respect to the selection of the $\mathbf{p}_i$ values, as long as the set of vectors $\boldsymbol{\phi}(\mathbf{p}_i)$ provide a sufficient[4] base in F. This observation is in agreement with Rosenblat's preliminary work on Perceptron training [41], and this principle has also served as an inspiration for Extreme Learning approaches, where the first layers of neural networks are randomly chosen [22, 23]. Taking all this into account, we propose to randomly generate the $\mathbf{p}_i$ pre-image values, which also helps to preserve the confidentiality of the data, since no training pattern is used as an architectural part of the final model. Therefore, in what follows, we will assume that the optimal hyperplane defined by $\mathbf{w}$ can be approximated by a linear combination of P projected pre-images, so that the classifier becomes

[3]It is important to explicitly define the kernel pre-images $\mathbf{p}_j$, so that dot products in F can be computed using the kernel function, this is, $\langle \boldsymbol{\phi}(\mathbf{x}_i), \boldsymbol{\phi}(\mathbf{p}_j) \rangle = k(\mathbf{x}_i, \mathbf{p}_j)$.

[4]Even when F has infinite dimension, it is possible to exactly represent the projections of the N_T training patterns using a base of size N_T. Also, SVMs only need to represent in F the SVs, so the base size is upper bounded by the number of SVs. Interestingly, depending on the kernel hyperparameters, the rank of the projected training patterns can even be smaller, obtaining good approximations even with base sizes much smaller than the number of SVs.

$$o_j = sign(f(\mathbf{x}_i, \boldsymbol{\beta}, b)), \tag{5}$$

$$f(\mathbf{x}_i, \boldsymbol{\beta}, b) = \sum_{i=1}^{P} \beta_i \boldsymbol{\phi}(\mathbf{p}_i)^T \boldsymbol{\phi}(\mathbf{x}_j) + b = \sum_{i=1}^{P} \beta_i k(\mathbf{p}_i, \mathbf{x}_j) + b. \tag{6}$$

Then the functional to be minimized (Equation (1)) can be transformed into

$$L_{WLS} = \frac{1}{2}\boldsymbol{\beta}^T\boldsymbol{\beta} + \frac{1}{2}\sum_{i=1}^{N_T} a_i e_i^2, \tag{7}$$

where

$$e_i = y_i - f(\mathbf{x}_i, \boldsymbol{\beta}, b) \tag{8}$$

and a_i are weighting values to be computed as

$$a_i = \begin{cases} 0, & e_i y_i < 0 \\ \frac{2C}{e_i y_i}, & e_i y_i \geq 0. \end{cases} \tag{9}$$

Further details about the derivation of expressions (7) and (9) can be found in [34, 35]. The minimization of the functional in Equation (7) (for fixed a_i) accepts a closed form and is equivalent to solving the following system of linear equations:

$$\begin{bmatrix} \mathbf{K}^T\mathbf{D}_a\mathbf{K} + \mathbf{K}_p & \mathbf{K}^T\mathbf{D}_a\mathbf{1} \\ \mathbf{1}^T\mathbf{D}_a\mathbf{K} & \mathbf{1}^T\mathbf{D}_a\mathbf{1} \end{bmatrix} \begin{bmatrix} \boldsymbol{\beta} \\ \mathrm{b} \end{bmatrix} = \begin{bmatrix} \mathbf{K}^T\mathbf{D}_a\mathbf{y} \\ \mathbf{1}^T\mathbf{D}_a\mathbf{y} \end{bmatrix}, \tag{10}$$

where $(\mathbf{K})_{i,j} = k(\mathbf{x}_i, \mathbf{p}_j)$, $(\mathbf{K}_p)_{i,j} = k(\mathbf{p}_i, \mathbf{p}_j)$, $\mathbf{y} = [y_1, \ldots, y_P]^T$, $\mathbf{1} = [1, \ldots, 1]^T$, and $\mathbf{D}_a = diag\{a_i\}$. To ease notation, we will group $\boldsymbol{\beta}$ and b into a single vector and define the following variables:

$$\ddot{\mathbf{K}} = [\mathbf{K}|\mathbf{1}]; \quad \ddot{\mathbf{K}}_p = \begin{bmatrix} \mathbf{K}_p & \mathbf{0} \\ \mathbf{0}^T & 0 \end{bmatrix}; \quad \ddot{\boldsymbol{\beta}} = \begin{bmatrix} \boldsymbol{\beta} \\ \mathrm{b} \end{bmatrix} \tag{11}$$

such that we can represent Equation (10) more compactly:

$$\left[\ddot{\mathbf{K}}^T\mathbf{D}_a\ddot{\mathbf{K}} + \ddot{\mathbf{K}}_r\right]\ddot{\boldsymbol{\beta}} = \left[\ddot{\mathbf{K}}^T\mathbf{D}_a\mathbf{y}\right]. \tag{12}$$

Therefore, an iterative two-step procedure can be applied to minimize Equation (7) by first updating a_i values using Equation (9), and secondly finding the optimal weights by solving Equation (12) and repeating until a given stopping criterion is met. The convergence properties of this **Iterated Re-Weighted Least Squares** (**IRWLS**) based optimization procedure have been shown in [12, 38]. The proposed BDSVM approach has the same convergence properties, since we also rely on the IRWLS optimization approach. Therefore, the algorithm for solving the BSVM functional using IRWLS can be summarized in Algorithm 1, where we obtain the new weights in step n using a convex combination between the current solution and the new estimated weights. If we set $\lambda = 0$, we are completely forgetting about the previous weights in each step. By using $0 < \lambda < 1$, we get a smoothed convergence. Furthermore, if we have a local validation set, it is possible to perform a line search on the λ parameter so that we obtain the optimal update (the one producing the best performance; AUC in the validation set in this case) at every epoch. In our experiments, we have used line search optimization and the stopping criterion is achieved when the weights do not change by more than a given percentage between two consecutive epochs, e.g., $\|\ddot{\boldsymbol{\beta}}^{(n)} - \ddot{\boldsymbol{\beta}}^{(n-1)}\|_2 / \|\ddot{\boldsymbol{\beta}}^{(n-1)}\|_2 < \eta$.

ALGORITHM 1: Budget Support Vector Machine (batch/centralized training)

Training data: N_T training pairs $\{\mathbf{x}_i, y_i\}_{i=1}^{N_T}$, rowwise arranged in matrix form as $\mathbf{X}, \mathbf{y}$.
Initialization: $\ddot{\boldsymbol{\beta}}^{(0)} = \mathbf{0}$
Pre-image generation: randomly generate P pre-image vectors $\mathbf{p}_j$.
Pre-image kernel computation: Compute $\ddot{\mathbf{K}}_p = \begin{bmatrix} \mathbf{K}_p & \mathbf{0} \\ \mathbf{0}^T & 0 \end{bmatrix}$, where $(\mathbf{K}_p)_{i,j} = k(\mathbf{p}_i, \mathbf{p}_j)$
Compute and store the kernel matrix for train data: $\ddot{\mathbf{K}} = [\mathbf{K}|\mathbf{1}]$;, where $(\mathbf{K})_{i,j} = k(\mathbf{x}_i, \mathbf{p}_j)$
repeat
  Compute errors on training data:
  - $\mathbf{e} = \mathbf{y} - < \ddot{\mathbf{K}}, \ddot{\boldsymbol{\beta}}^{(n)} >$

  Compute weighting values a_i:
  - $a_i = 0$ if $e_i y_i < 0$; $a_i = \frac{2C}{e_i y_i}$ if $e_i y_i \geq 0$

  Define $\mathbf{D}_a = diag\{a_i\}$ and solve the WLS Problem to obtain the new weights:
  - $\ddot{\boldsymbol{\beta}}_{new} = ([\ddot{\mathbf{K}}^T \mathbf{D}_a \ddot{\mathbf{K}} + \ddot{\mathbf{K}}_p])^{-1} [\ddot{\mathbf{K}}^T \mathbf{D}_a \mathbf{y}]$

  Update the model using either fixed λ or line search using a validation set:
  - $\ddot{\boldsymbol{\beta}}^{(n+1)} = \lambda \ddot{\boldsymbol{\beta}}^{(n)} + (1 - \lambda) \ddot{\boldsymbol{\beta}}_{new}$

until the chosen stopping criterion is met

2.2 BDSVM for a Federated Environment

We propose to extend the BSVM formulation in Section 2.1 to operate in a FL environment, ultimately obtaining the Budget Distributed SVM approach, BDSVM. Therefore, the aggregator shares the model with the workers and they compute the kernel matrices in Equation (12) using both their local training data and the weighting values a_i derived from the global model shared by the aggregator, to finally send them back to the aggregator. Based on this description, BDSVM can be thought of as a Federated approach, in the sense that bulky computations ($\ddot{\mathbf{K}}^T \mathbf{D}_a \ddot{\mathbf{K}}$ and $\ddot{\mathbf{K}}^T \mathbf{D}_a \mathbf{y}$) are done in the workers and the updated global model is obtained at the aggregator. BDSVM differs slightly from FL in that instead of transmitting the model parameters, we need to pass the information represented by $\ddot{\mathbf{K}}^T \mathbf{D}_a \ddot{\mathbf{K}}$ and $\ddot{\mathbf{K}}^T \mathbf{D}_a \mathbf{y}$. As we will see in the experimental section, the potentially greater amount of information to be transmitted in each epoch is compensated by a convergence in fewer steps so that, in global terms, the total amount of information transmitted turns out to be less than in FedAvg, which in the non-ID case requires many epochs to converge to a competitive solution.

Therefore, worker m has a subset of training patterns $\{\mathbf{x}_i^m, y_i^m\}_{i=1}^{N_T^m}$ such that assuming it knows $\mathbf{p}_j$ and can update $\mathbf{D}_a^m = diag\{a_i^m\}$ using the received global model, it can compute locally

$$(\mathbf{K}_m)_{i,j} = k(\mathbf{x}_i^m, \mathbf{p}_j), \tag{13}$$

$$\ddot{\mathbf{K}}_m = [\mathbf{K}_m|\mathbf{1}], \tag{14}$$

$$\mathbf{C}_m = \ddot{\mathbf{K}}_m^T \mathbf{D}_a^m \ddot{\mathbf{K}}_m, \tag{15}$$

$$\mathbf{d}_m = \ddot{\mathbf{K}}_m^T \mathbf{D}_a^m \mathbf{y}^m \tag{16}$$

and transmit $\mathbf{C}_m$, $\mathbf{d}_m$ to the aggregator. Once the aggregator receives the contributions from all the workers, it can find the new model weights by solving

$$\left(\sum_{m=1}^{M} \mathbf{C}_m + \ddot{\mathbf{K}}_p\right)\ddot{\boldsymbol{\beta}} = \sum_{m=1}^{M} \mathbf{d}_m. \tag{17}$$

An intrinsic advantage of the procedure described above is that it is robust against non-ID data partitioning, since the distributed functional in Equation (17) is formally equivalent to the centralized one in Equation (12). In this sense, the same good convergence properties observed in the centralized case also apply to the federated version, since the only difference is how $\mathbf{C}_m$ and $\mathbf{d}_m$ are computed in a distributed setup. In the next section, we will analyze how much information needs to be transmitted and we will also propose some techniques to reduce the number of bytes transmitted, to finally obtain the BDSVM algorithm.

2.3 BDSVM Data Transmission Cost Reduction

Any FL schema involves two dataflows: the aggregator broadcasts the updated model to the M workers, and the workers send back the updated local model to the aggregator. If we assume a scenario with M workers and an MLP architecture with N_F inputs and P neurons in the hidden layer, we need to transmit in each epoch a total number of $2M((N_F+1)P+(P+1)))$ floats. On the other hand, in the BDSVM case when we use a model with the same complexity (P pre-images), the aggregator needs to send the model to each worker, and the workers need to send $\mathbf{C}_m$ and $\mathbf{d}_m$ to the aggregator, representing a total number of $M(N_F P+(P+1)+(P+1)^2+(P+1))$ floats per epoch. However, the structure of the model and the characteristics of the data matrices to be exchanged in BDSVM allow the total number of transmitted bytes to be further reduced. First, the P pre-image vectors in BDSVM do not change over time, and therefore only need to be transmitted once. Also, they can be generated locally on each worker if we take advantage of the pseudorandom number generation properties, i.e., we only need to send once the seed used to start the pseudo-random generator, so the transmission reduces to $1+M((P+1)+(P+1)^2+(P+1))$. Additionally, the $\mathbf{C}_m$ matrix is always symmetric, so we only need to send its diagonal plus the upper triangle, resulting in a new reduced transmission of $1+M((P+1)+(P+1)(P+2)/2+(P+1))$. Finally, suppose that under certain circumstances $g = rank(\mathbf{C}_m)$ is very small, in which case it might be very advantageous to compute a QR decomposition of $\mathbf{C}_m$:

$$(\mathbf{C}_m)_{(P+1)\times(P+1)} = (\mathbf{Q}_m)_{(P+1)\times g}(\mathbf{R}_m)_{g\times(P+1)}. \tag{18}$$

We could directly compute the rank g at every step, but alternatively we can easily obtain an upper bound. If we rewrite $\mathbf{C}_m$ as follows:

$$\mathbf{C}_m = \sum_{i=1}^{N_T} a_i \mathbf{k}_i^m \left(\mathbf{k}_i^m\right)^T, \tag{19}$$

where $(\mathbf{k}_i^m)_{(0)} = 1$, $(\mathbf{k}_i^m)_{(j+1)} = k(\mathbf{x}_i^m, \mathbf{r}_j)$. It is clear that the rank of $\mathbf{C}_m$ is upper bounded by the number of training patterns. Also, only SVs (training patterns with nonzero a_i) contribute to the sum in Equation (19), thus a tighter upper bound on the rank of $\mathbf{C}_m$ is determined by the number of SVs. Therefore, in the case where the number of workers is very large and each worker has very few training patterns or support vectors, the rank can be very small and hence $\mathbf{C}_m$ is very compactable. If none of the training patterns in a given worker becomes SV, that worker can simply return an empty matrix. Therefore, we can either compute $g = rank(\mathbf{C}_m)$ or directly use N_V as an upper bound (we will use the upper bound $g = N_V$ in the experiments). It is more efficient to send

the QR decomposition $\mathbf{Q}_m$ and $\mathbf{R}_m$ than the upper triangle part of $\mathbf{C}_m$ whenever

$$2Pg < (P+1)(P+2)/2. \tag{20}$$

2.4 Risk Analysis

We present here a preliminary analysis of the robustness of the proposed algorithm with respect to inference attacks (information leakage from the information exchanged during training and also from the trained model itself). The variety of potential attacks is so great that it exceeds the scope of this article to carry out an exhaustive test of all possible attack models existing in the literature, which is left as future work. However, we carry out here a preliminary security analysis of the proposed algorithm, in the hope that it will encourage other researchers to develop "ad hoc" inference attacks especially targeting the proposed BDSVM scheme, to ultimately learn about its potential robustness under a wider variety of scenarios and specific attackers.

Our analysis will cover two possible leak situations. The first one takes place during the training process; the aggregator receives some aggregated measures/statistics from the workers and we want to know to what extent the training data can be inferred from the received parameters. Some recent work shows that under certain circumstances it is possible to reconstruct training patterns from the gradient information, especially when individual gradients with respect to each training pattern are available [65]. However, this situation rarely occurs in practice, since the information that is exchanged usually consists of model parameters or averaged magnitudes, as in our case. First of all, note that we are not sending gradients and that the received information at the aggregator from worker "m" is represented by $\mathbf{C}_m$ (Equation (15)) and $\mathbf{d}_m$ (Equation (16)), which are very convoluted and averaged functions of the training patterns. For example, the (i, j) value in $\mathbf{C}_m$ is computed as a weighted sum of Gaussian kernels: $\mathbf{C}_m(i,j) = \sum_{n=0}^{N} k(\mathbf{p}_i, \mathbf{x}_n) a_n k(\mathbf{p}_j, \mathbf{x}_n)$, where the weights a_n are a non-differentiable non-linear function of the training data, as indicated by Equation (9). Therefore, it is not possible to try to infer the values of the training data using a gradient descent approach as in [65], since the equations include a non-differentiable factor. Thus, the aggregator receives up to $P(P-1)/2$ independent equations, but the number of unknowns is also huge (number of training patterns $\times$ number of input features). Furthermore, these equations include a non-linear non-invertible relationship between the values in $\mathbf{C}_m, \mathbf{d}_m$ and the kernels $k(\mathbf{p}_j, \mathbf{x}_n)$ such that it is not possible to solve the equations by direct matrix inversion, as in an—overdetermined—linear case. Even in the hypothetical case that someone is able to obtain all the kernel values, there is no unique relationship to the input values, since due to the spherical symmetry of the Gaussian kernel used here, there are an infinite number of potential points $\mathbf{x}_n$ yielding the same kernel value $k(\mathbf{p}_j, \mathbf{x}_n)$ (all those points at the same distance from $\mathbf{p}_j$ in a N_F dimensional space). As a general conclusion, we see that it is not possible to obtain the training data from the $\mathbf{C}_m$ and $\mathbf{d}_m$ values.

A second leak risk analyzed here is the well-known **Membership Inference Attack** (**MIA**), which attempts to identify whether a particular training pattern has been used during the training phase by exploring the model outputs. MIA is based on the assumption that the trained model is somehow overfitting the training data [47], so that by inspecting the confidence level obtained for a given input pattern, we can identify wether it was included in the training set (higher confidence than other patterns). As shown in [43], there is a direct relationship between the level of model overfitting and the effectiveness of membership inference attacks. This conclusion has also been presented in [42], where it is recognized that minimizing overfitting is the best strategy against MIA. So, if we can show that the described BDSVM algorithm produces correctly regularized models, we are safe in the sense that their outputs cannot be used to identify the training patterns (all the outputs have the same confidence, regardless of whether or not they were used during

ALGORITHM 2: Budget Distributed Support Vector Machine (BDSVM) *at the aggregator*

1: **Weights Initialization:** $\ddot{\boldsymbol{\beta}}^{(0)} = \mathbf{0}$
2: **Pre-image computation:** The aggregator generates a random seed S, and computes P pre-image vectors $\mathbf{p}_j$ with a pseudorandom number generator using S as a seed.
3: **Pre-image kernel computation:** Compute $\ddot{\mathbf{K}}_p = \begin{bmatrix} \mathbf{K}_p & \mathbf{0} \\ \mathbf{0}^T & 0 \end{bmatrix}$, where $(\mathbf{K}_p)_{i,j} = k(\mathbf{p}_i, \mathbf{p}_j)$
4: **repeat**
5: At epoch n, the aggregator broadcasts the model weights $\ddot{\boldsymbol{\beta}}^{(n)}$ to the workers.
6: The aggregator receives the contribution from worker m, $\mathbf{d}_m$, and $(\mathbf{Q}_m, \mathbf{R}_m)$ or $\mathbf{U}_m$ and uses it to rebuild $\mathbf{C}_m$:
- If the QR decomposition was used: $\mathbf{C}_m =< \mathbf{Q}_m, \mathbf{R}_m >$.
- If the upper triangular decomposition was used: $\mathbf{C}_m = \mathbf{U}_m + \mathbf{U}_m^T - \frac{1}{2} diag(\mathbf{U}_m)$.

7: The aggregator waits a maximum time $T_{timeout}$ to receive the updates from the workers and aggregates all the received contributions to obtain $\ddot{\boldsymbol{\beta}}_{new}$:
- $\ddot{\boldsymbol{\beta}}_{new} = (\sum_{m=1}^{M} \mathbf{C}_m + \ddot{\mathbf{K}}_p)^{-1} \sum_{m=1}^{M} \mathbf{d}_m$.

8: The aggregator updates the model:
- $\ddot{\boldsymbol{\beta}}^{(n+1)} = \lambda \ddot{\boldsymbol{\beta}}^{(n-1)} + (1-\lambda)\ddot{\boldsymbol{\beta}}_{new}$

9: **until** the chosen convergence criterion is met ($||\ddot{\boldsymbol{\beta}}^{(n+1)} - \ddot{\boldsymbol{\beta}}^{(n)}||_2 / ||\ddot{\boldsymbol{\beta}}^{(n)}||_2 < \eta$).

training). A quantitative measure of overfitting is given by the performance gap, the difference between the performance of the trained model on its training and validation/test data [18, 47]. The robustness of the proposed BDSVM algorithm has been evaluated experimentally and the results are presented in Section 3.4, concluding that BDSVM is secure from a MIA point of view.

In summary, the full BDSVM algorithm is described in Algorithms 2 (aggregator part) and 3 (worker/participant part). We have also included a timeout parameter ($T_{timeout}$), which is the maximum time that the aggregator waits for the contributions from the workers before updating the model. This parameter is required to deal with dropout or straggler cases as described in more detail in Section 3.3. In the experimental section, we will illustrate/benchmark in depth the features of the proposed algorithms in various non-ID data partition scenarios.

3 EXPERIMENTS

In this section, we compare the performance of the proposed BDSVM algorithm using synthetic examples and a selection of public domain datasets for binary classification. First of all, we illustrate how the decision boundary obtained by BDSVM shows the maximum margin characteristic, typical of SVMs in separable problems. We will also compare the performance and transmission cost of a BDSVM with respect to a Neural Network trained using FedAvg [31] (typical setup for federated machine learning) using scenarios with an increasing amount of virtual workers and a real deployment in a FL platform with real communications. We have also included some benchmarks for the dropout/straggler scenarios and evaluated BDSVM's resiliency against membership inference attacks.

3.1 Illustrative Examples on Synthetic 2D Datasets

We evaluate here BDSVM on 2D synthetic datasets, so that we can observe the resulting decision boundaries. In Figures 1 and 2, we have defined two separable datasets with different complexity

ALGORITHM 3: Budget Distributed Support Vector Machine (BDSVM) *at worker "m"*

1: **Training data:** worker m has N_T^m patterns $\{\mathbf{x}_i^m, y_i^m\}_{i=1}^{N_T^m}$, rowwise arranged as $\mathbf{X}^m, \mathbf{y}^m$.
2: **Initialization:** the worker receives the seed S and generates the P pre-images $\mathbf{p}_j$ using a pseudorandom number generator initiated by S.
3: **Compute and store the kernel matrix for train data:** the worker computes and stores the training Kernel matrix $\ddot{\mathbf{K}}_m = [\mathbf{K}_m|\mathbf{1}]$;, where $(\mathbf{K}_m)_{i,j} = k(\mathbf{x}_i^m, \mathbf{p}_j)$.
4: **repeat**
5: The worker receives the model weights $\ddot{\boldsymbol{\beta}}^{(n)}$ and computes the errors on its training data:
- $\mathbf{e}_m = \mathbf{y}_m - < \ddot{\mathbf{K}}_m, \ddot{\boldsymbol{\beta}}^{(n)} >$.

6: The worker computes the weighting values a_i^m:
- $a_i^m = 0$ if $e_i^m y_i^m < 0$; $a_i^m = \frac{2C}{e_i^m y_i^m}$ if $e_i^m y_i^m \geq 0$

and builds $D_a^m = diag(a_i^m)$. N_v^m is the number of SVs (nonzero a_i^m values).
7: The worker computes $\mathbf{C}_m$ and $\mathbf{d}_m$:
- $\mathbf{C}_m = \ddot{\mathbf{K}}_m^T \mathbf{D}_a^m \ddot{\mathbf{K}}_m$
- $\mathbf{d}_m = \ddot{\mathbf{K}}_m^T \mathbf{D}_a^m \mathbf{y}^m$

8: Compute $g = rank(\mathbf{C}_m)$ or use the upper bound $g = N_V$ and compact $\mathbf{C}_m$:
- If $2Pg < (P+1)(P+2)/2$, send back $\mathbf{d}_m$ and $(\mathbf{Q}_m, \mathbf{R}_m) = QR(\mathbf{C}_m)$.
- If $2Pg > (P+1)(P+2)/2$, send back $\mathbf{d}_m$ and $\mathbf{U}_m$ the upper triangle of $\mathbf{C}_m$

9: **until** the training process is finished by the aggregator.

and used a non-ID data partition into four workers so that the workers only have data covering a part of the input space and some of them only have samples from one class. It can be seen that BDSVM is able to find maximum margin solutions, providing boundaries equivalent to the centralized ones. In Figure 3, we have randomly generated a non-separable dataset, repeated an analogous setup, and also found that BDSVM is able to provide a good global classification boundary that solves the classification problem defined by the distributed dataset.

3.2 Performance on Real-World Datasets

Here, we compare the performance of BDSVM with a Neural Network in a non-ID FL setting using the real-world datasets listed in Table 1. The MNIST database of handwritten digits ("Mnist") [27] has been converted to the binary task of distinguishing between even and odd numbers. The phishing dataset is all about predicting malicious websites [32]. The Census Income Data Set ("Income") is intended to predict if income exceeds $50K\$/yr$ based on census data [10]. The ijcnn1 dataset was used in a ML challenge during IJCNN 2001 [37] and "webspam" is a corpus intended to detect spam websites [54]. In Table 1, we indicate, for each dataset, the number of patterns for training, validation, and testing, as well as the number of input features. We also show the size of the model used (number of kernels in the case of SVM and number of neurons (ReLU activations) in the case of MLP), as well as the C and σ hyperpameters for the SVM case. These model hyperparameters have been obtained using error cross-validation.

The training data is distributed across an increasing number of workers,[5] so that each worker only has instances of a single class (non-ID scenario). We have used virtual workers here, to be

[5]In all experiments, all training data was used; the only difference is how it is distributed across workers, so we should be able to achieve the same performance even when the dataset is distributed across many participants.

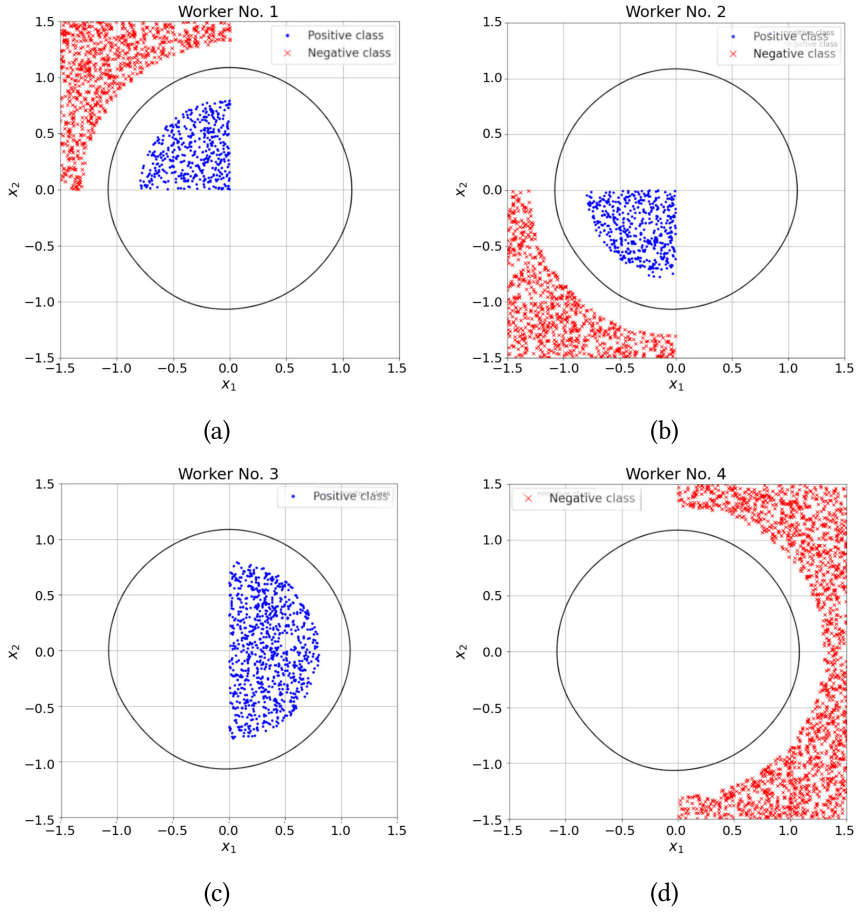


Fig. 1. BDSVM result in a separable circular problem. The global learned model and local training data at each worker in a non-ID setting are shown.

able to simulate scenarios with up to 2,048 workers. In the next section, we will complete the experiments with a real communication scenario using the MUSKETEER platform where each worker runs on a separate core/thread and the transmission is done through a cloud service, so we can measure the actual execution times. For now we will concentrate on measuring the final performance achieved. The AUC, Area under **Receiving Operation Curve** (**ROC**), is used as a performance measure, as it is not dependent on any specific threshold selection. We also calculate in this experiment the number of Mbytes exchanged in each case during the training process. The results are shown in Figures 4–8, where we have not included the experiments that took more than 48 hours to converge, as is the case of many MLP-FedAvg experiments with a large number of workers. In all of these figures, we show the AUC for both BDSVM, BDSVM-QR (the BDSVM algorithm when the QR compression method is used) and MLP-FedAvg in (a) and the number of Mbytes transmitted in (b). Essentially, BDSVM and BDSVM-QR produce the same AUC results, since the same information matrices are used during training, but BDSVM-QR can reduce the amount of information transmitted when the number of workers is large. We observe that the AUC achieved by BDSVM/BDSVM-QR is comparable to that of MLP-FedAvg for the case of one worker

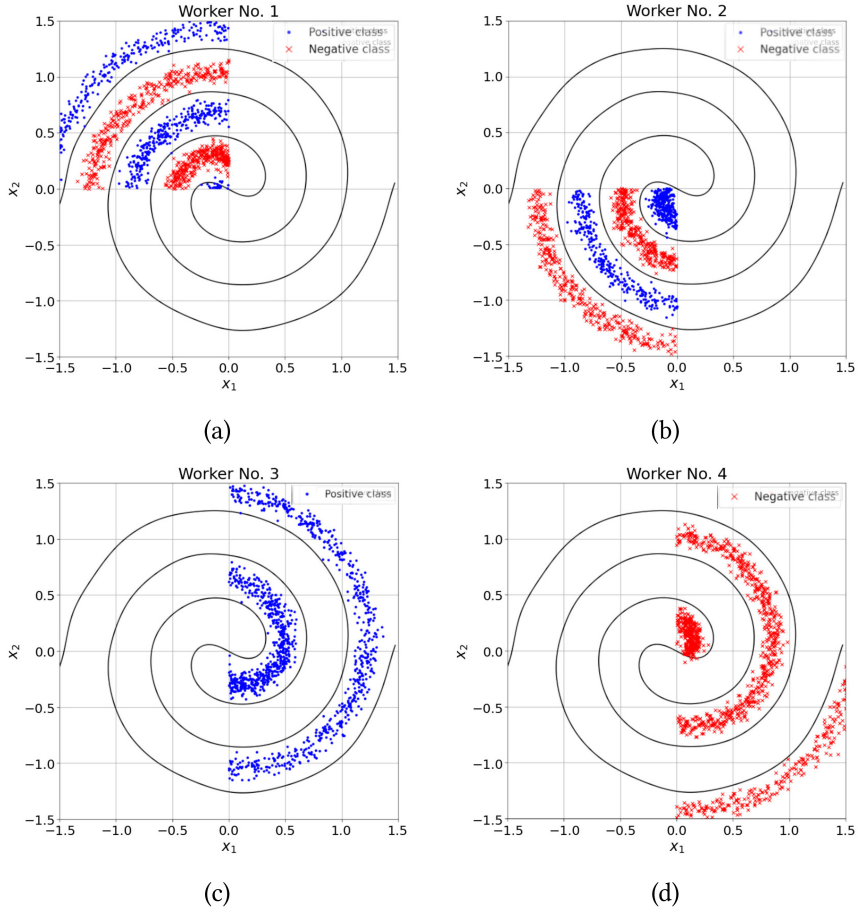


Fig. 2. BDSVM result in a separable two-spirals problem. The global learned model and the local training data at each worker in a non-ID setting are shown.

(situation equivalent to the centralized solution). However, with a larger number of workers, the performance of MLP-FedAvg tends to decrease in most cases, while the performance of BDSVM is maintained even for a large amount of workers. Regarding Mbytes transmitted, we note that due to the large number of epochs it takes for MLP-FedAvg to converge in the non-ID setting, it actually transmits more information than BDSVM (27 times more, on average for all datasets and experiments). Also, when the number of workers is high, BDSVM benefits from the compact QR-decomposition approach explained in Section 2.3; these results are marked as "BDSVM-QR" in Figures 4—8(b).

3.3 Performance in the Presence of Dropout or Straggler Workers

As already mentioned above, the work presented here has been developed in the context of the MUSKETEER project, which is an Industrial Data Platform (also known as Enterprise Federated Learning). Under this assumption, the number of "enterprise" participants is expected to be moderate and the amount of information held by each, large. The offline/dropout events should not occur frequently, as we want to ensure that as much information as possible is used to produce the

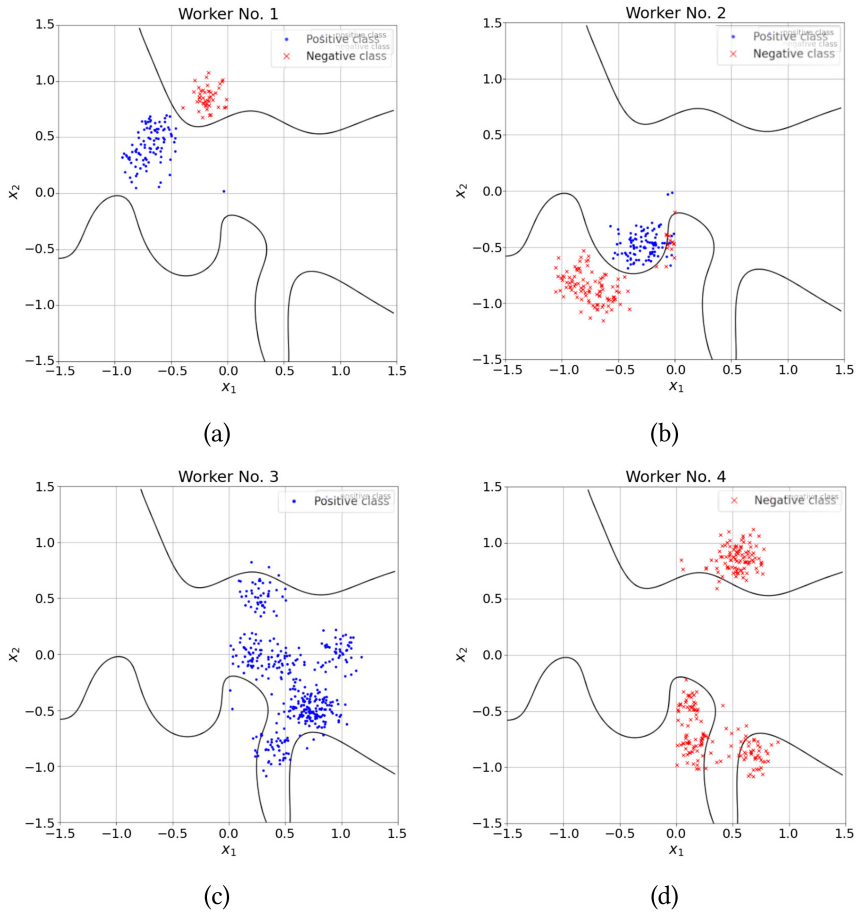


Fig. 3. BDSVM result on random data. The global learned model and the local training data at each worker in a non-ID setting are shown.

Table 1. Datasets Used for Benchmarking

Dataset	Ntr	Nval	Ntst	Nf	Size	C	σ
Mnist	50,000	10,000	10,000	784	5,500	10	5.6
Phishing	9,000	1,055	1,000	68	1,500	120	0.58
Income	26,049	6,512	16,281	107	9,000	100	20.7
ijcnn1	35,000	14,990	91,701	22	2,000	5	1.88
webspam	200,000	50,000	100,000	254	2,000	2,000,000	4.8

final model. Take, for example, the case of several large companies that are willing to join forces to build an improved model and decide to cooperate. It doesn't make much sense for one or more of them to not respond during the training process (in which case, the training would be postponed to fit all of their "agendas"). Also, if one of the participants is slower than the other in processing its data and producing a contribution (straggler), the waiting times must be adapted to avoid loss of information. However, since the proposed approach could also be extended to the **Personal Data Platform** (**PDP**) case (e.g., millions of participants, each providing little data), we include here some experiments to illustrate the effects of missing contributions. There are several situations

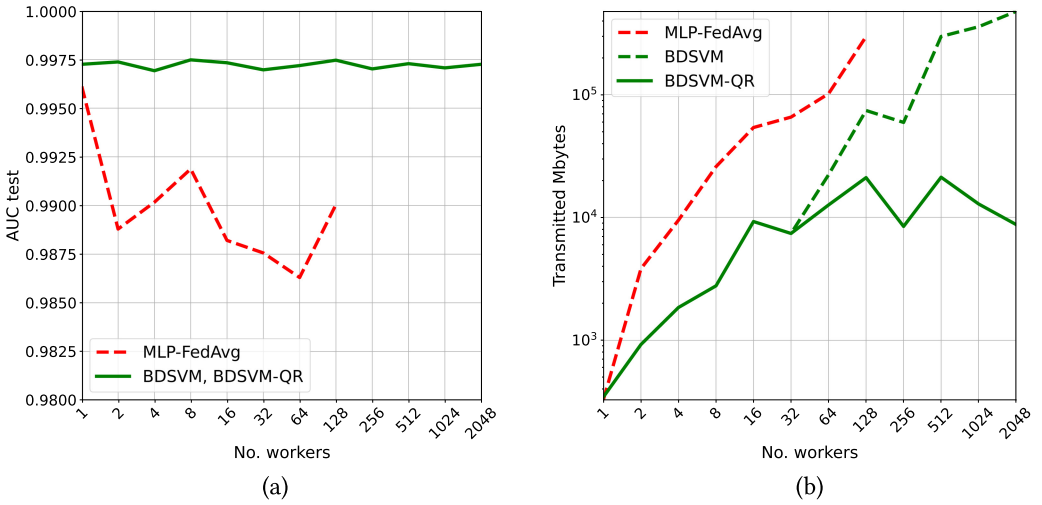


Fig. 4. Result of BDSVM result on a real dataset (even/odd MNIST). AUC performance on the test set is shown for MLP-FedAvg, BDSVM, and BDSVM-QR in (a). The total Mbytes transmitted are shown in (b).

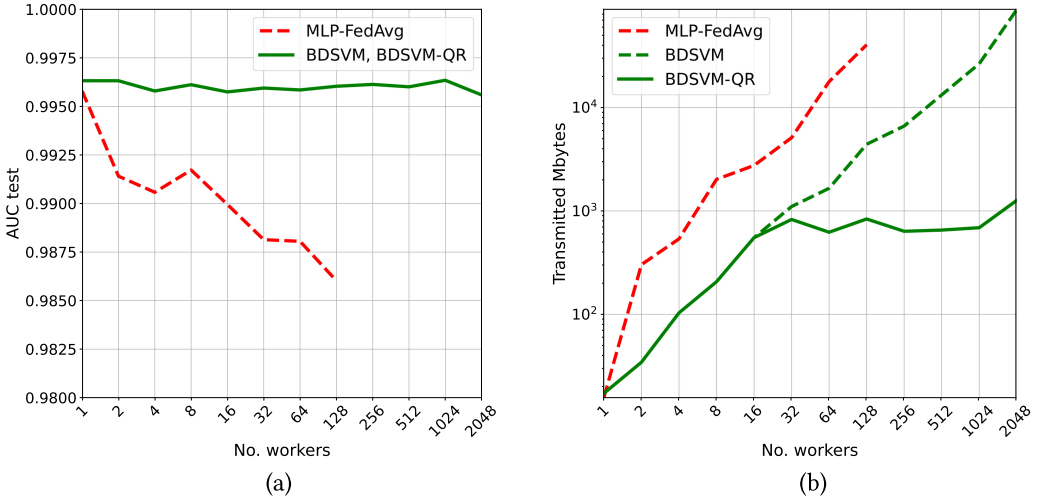


Fig. 5. Result of BDSVM result on a real dataset (phishing). AUC performance on the test set is shown for MLP-FedAvg, BDSVM, and BDSVM-QR in (a). The total Mbytes transmitted are shown in (b).

that can produce the lack of response of a certain worker to the training process, mainly under the PDP assumption. One cause may be that the participant disconnects during the FL training process or is no longer able to respond (dropout), which could be the case if the worker is a mobile phone that runs out of battery or enters an area with low or null connectivity. Another reason for losing a contribution from a given worker could be that it takes too long to compute an answer due to variability in hardware (CPU, memory), network connection (3G, 4G, WiFi), or power (battery level), and the rest of the participants cannot wait for this "slow" worker (straggler). The number of scenarios and possibilities is enormous, and we will model these situations by uniting all the possible dropout/straggler causes using a single parameter p_{loss}, the probability of losing the

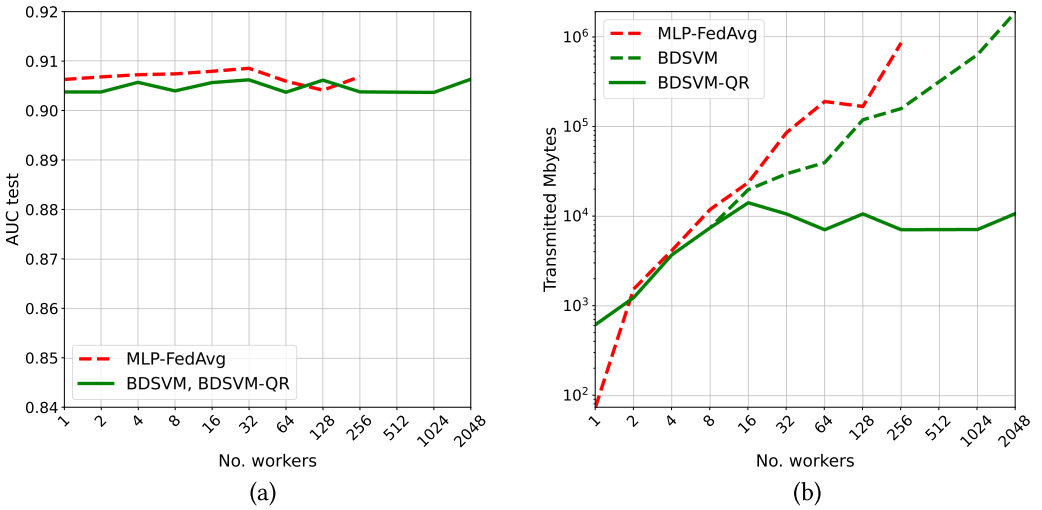


Fig. 6. Result of BDSVM result on a real dataset (Income). AUC performance on the test set is shown for MLP-FedAvg, BDSVM, and BDSVM-QR in (a). The total Mbytes transmitted are shown in (b).

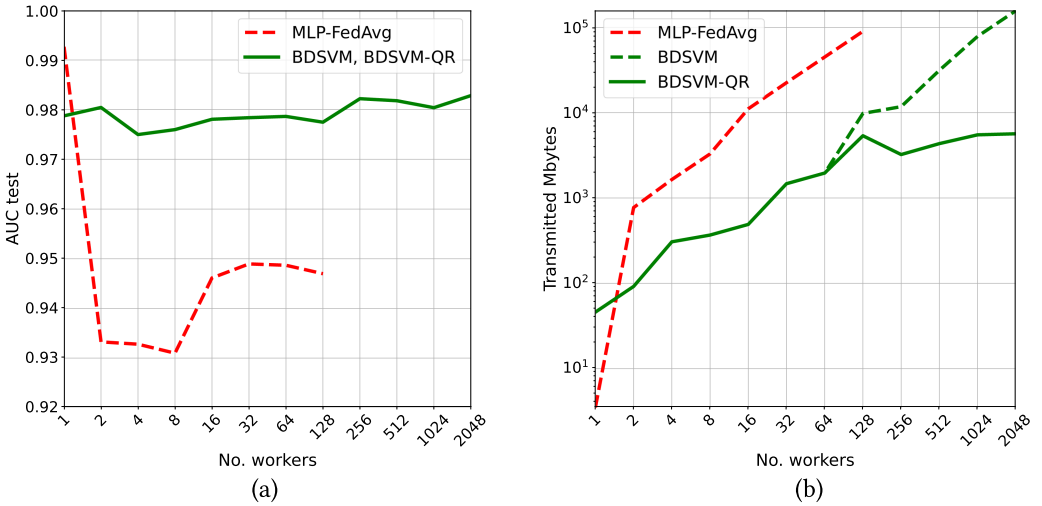


Fig. 7. Result of BDSVM result on a real dataset (ijcnn1). AUC performance on the test set is shown for MLP-FedAvg, BDSVM, and BDSVM-QR in (a). The total Mbytes transmitted are shown in (b).

update from a given worker. Therefore, we will assume that the training operation is as described in Algorithms 2 and 3, but the aggregator waits a maximum amount of time ($T_{timeout}$ parameter in Algorithm 2) to receive the responses of the workers, so that there is a non-zero probability that a worker's contribution is not available when the model needs to be updated at the aggregator (the maximum waiting time is reached).

We have run several experiments for the datasets in Table 1 for different values of p_{loss}; we have measured the resulting performance and plotted the corresponding results in Figures 9(a)–(f). The numbers near the curves indicate the percentage of training data that has been used during training, at least during one of the iterations of the algorithm. For example, in the case of MNIST

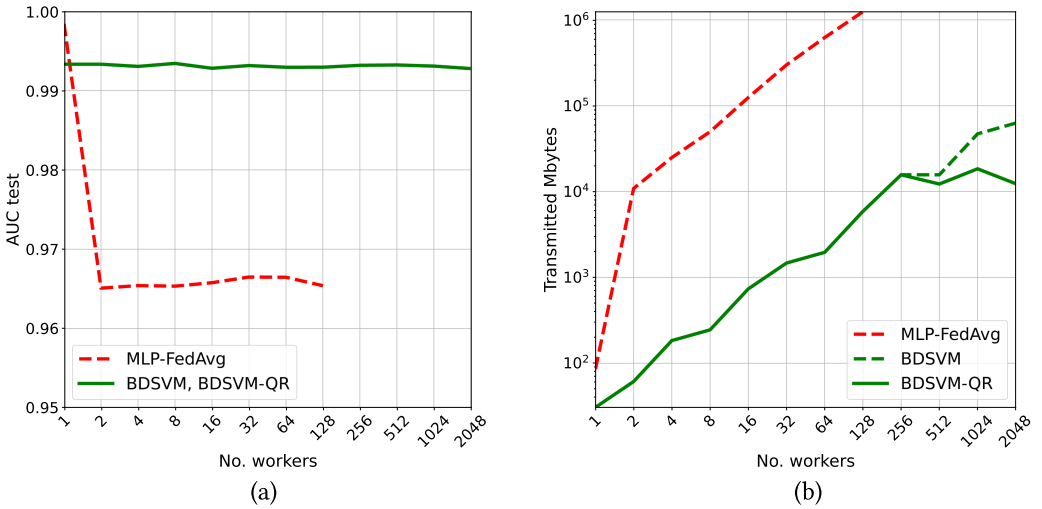


Fig. 8. Result of BDSVM result on a real dataset (webspam). AUC performance on the test set is shown for MLP-FedAvg, BDSVM, and BDSVM-QR in (a). The total Mbytes transmitted are shown in (b).

Table 2. Performance Gap Measurement

Dataset	AUC (train)	AUC (val)	AUC (tst)	Perf. gap (train-val)	Perf. gap (train-test)
Mnist	0.9981	0.9965	0.9970	0.16%	0.11%
Phishing	0.9946	0.9944	0.9947	0.03%	0.01%
Income	0.9072	0.9036	0.9037	0.39%	0.39%
ijcnn1	0.9893	0.9933	0.9833	0.41%	0.61%
webspam	0.9937	0.9932	0.9929	0.05%	0.08%

(Figure 9(a)), when $p_{loss} = 0.5$ we have a situation where half of the contributions are lost during the training process, but considering all the iterations, a total of 96% of the training data was at least used once. In general, we observe that the proposed algorithm is highly resilient against information loss. Basically, a small decrease in performance is observed for small values of p_{loss}. We observe that severe performance damage is not observed until losses are really high ($p_{loss} > 0.9$). Therefore, even for significant losses (p_{loss} around 0.1), the proposed algorithm is able to provide valid models without sacrificing much performance. The different shapes and slopes observed in Figures 9(a)–(f) reflect the amount of redundancy in the datasets, that is, how much information is lost in the training process when a fraction of the data is not used.

3.4 Robustness Against MIAs

It is well recognized in the MIA literature that the main factor facilitating successful attacks is model overfitting [43, 47]. One way to measure the level of overfitting is determined by the train-test (also train-validation) performance gap, this is, the difference in performance achieved on the train, validation, or test datasets. Getting much better performance on the train dataset than on the validation or test sets is a clear sign of overfitting. In Table 2, we illustrate the AUC values for every dataset, as well as the performance gap, in percentage.

We observe that the gap is always below 0.6% (0.2% on average), which is very small, indicating that the performance on the training, validation, and test datasets is very similar, there is no overfitting, and therefore MIA attacks can hardly succeed.

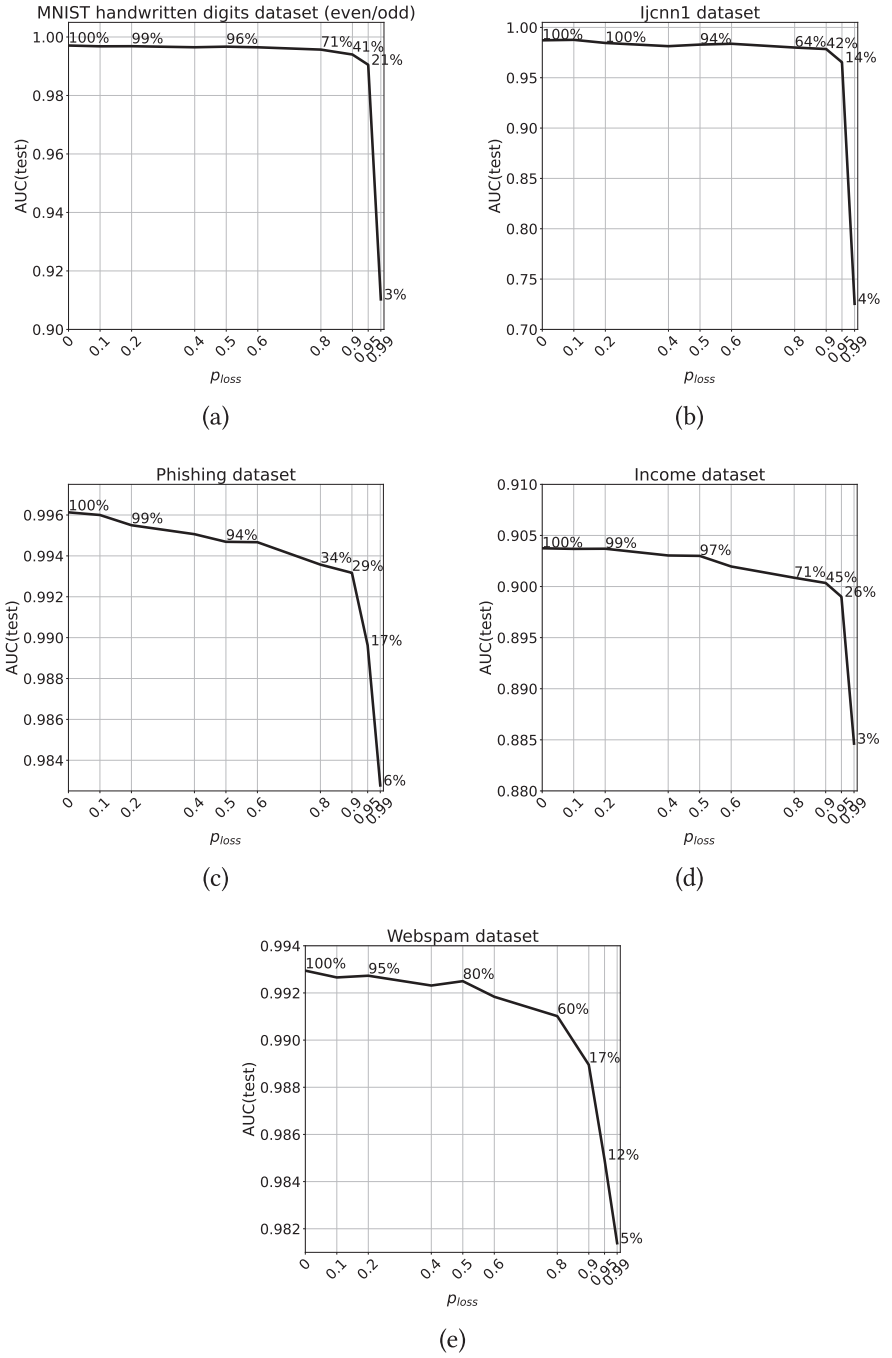


Fig. 9. Performance (AUC) evolution as a function of the loss probability p_{loss}.

We also evaluate here the "worst case" scenario, where the attack is implemented by an "omniscient" user who has access to the training and test datasets. In a general case, any attacker will have much less (even zero) access to the information used during training and will need to estimate it, so these attacks will be much less effective than the one presented here, designed with full access to the training data. Therefore, the attacker builds a new predictive model to discriminate between samples in the training set and samples in the validation/test sets, using the predictions of the BDSVM model as inputs, and with positive target if the pattern was used for training and negative otherwise. We have trained and evaluated those attack models, one for each dataset and BDSVM model, and the resulting ROC curves are shown in Figure 10. We note that in most cases AUC and accuracies are close to 0.5, indicating that the confidence level in the predictions is low and it is not possible to identify the training patterns with sufficient certainty using the BDSVM model outputs as input information. In the case of the ijcnn1 dataset, accuracy is 0.62, since the data itself has shown significant differences between training, validation, and test; that is, the small leak does not come from the trained model. We can conclude that the distribution of the model outputs is comparable for the training, validation, and test sets, so it is not possible to exploit that information to determine if a given pattern was used during training or not by exploring the model outputs. Therefore, the ability to predict the membership of a specific pattern to the training set from the model outputs is very limited, even for an "omniscient" attacker, so BDSVM models are safe from the MIA perspective.

3.5 Performance on MUSKETEER FL Platform

We will evaluate here the performance of MLP-FedAvg and BDSVM when using a real communication setup to transmit data between different processes/machines interconnected through a Cloud Messenger Service. The proposed algorithms have been implemented in the **MUSKETEER Machine Learning Library** (**MMLL**) and incorporated into the MUSKETEER platform, a privacy-preserving ML Industrial Data Platform that aims to operate under different Privacy Operation Modes designed to offer higher standards of security and privacy to the stakeholders of the platform, FL being one of the possibilities [1, 49]. The MUSKETEER platform architecture consists of a server side hosted in the cloud (comprising a RabbitMQ service plus a number of additional micro-services) and using message queues for the asynchronous exchange of information between the federation of aggregator and participants (the client side), which can be arbitrarily located geographically. This communication mechanism has been shown to be very secure and efficient, compared to other approaches that rely on less secure procedures or require opening communication ports, an approach that is generally not allowed by security firewalls [1]. The MUSKETEER client side provides an effective interface for the aggregator and participants to interact with the platform.

We have carried out the experiments using various machines[6] located at the Universidad Carlos III of Madrid, so we can test the real performance of a federated scenario with up to 32 workers (one separate core/thread each), communicating through cloud services located in Frankfurt. On average, the effective communication speed achieved in this scenario is approximately 2.7 Mbytes per second (average throughput or digital bandwidth capacity). There is no direct communication between local processes; all message exchange is done through the cloud service. We summarize the obtained results in Tables 3 and 4. As already noted in the previous subsection, MLP training under FedAvg in the non-ID setting has presented many convergence issues, leading to exceedingly long training times, as the number of local updates had to be greatly reduced,

[6]Dual Intel Xeon processor E5-2640 v4 running at 2.40 GHz with 10 cores/20 threads, 128 Gb of memory, 2×1 Gbit network interface. and 2×1 Tbyte hard disk, running a Gentoo distribution optimized for this architecture.

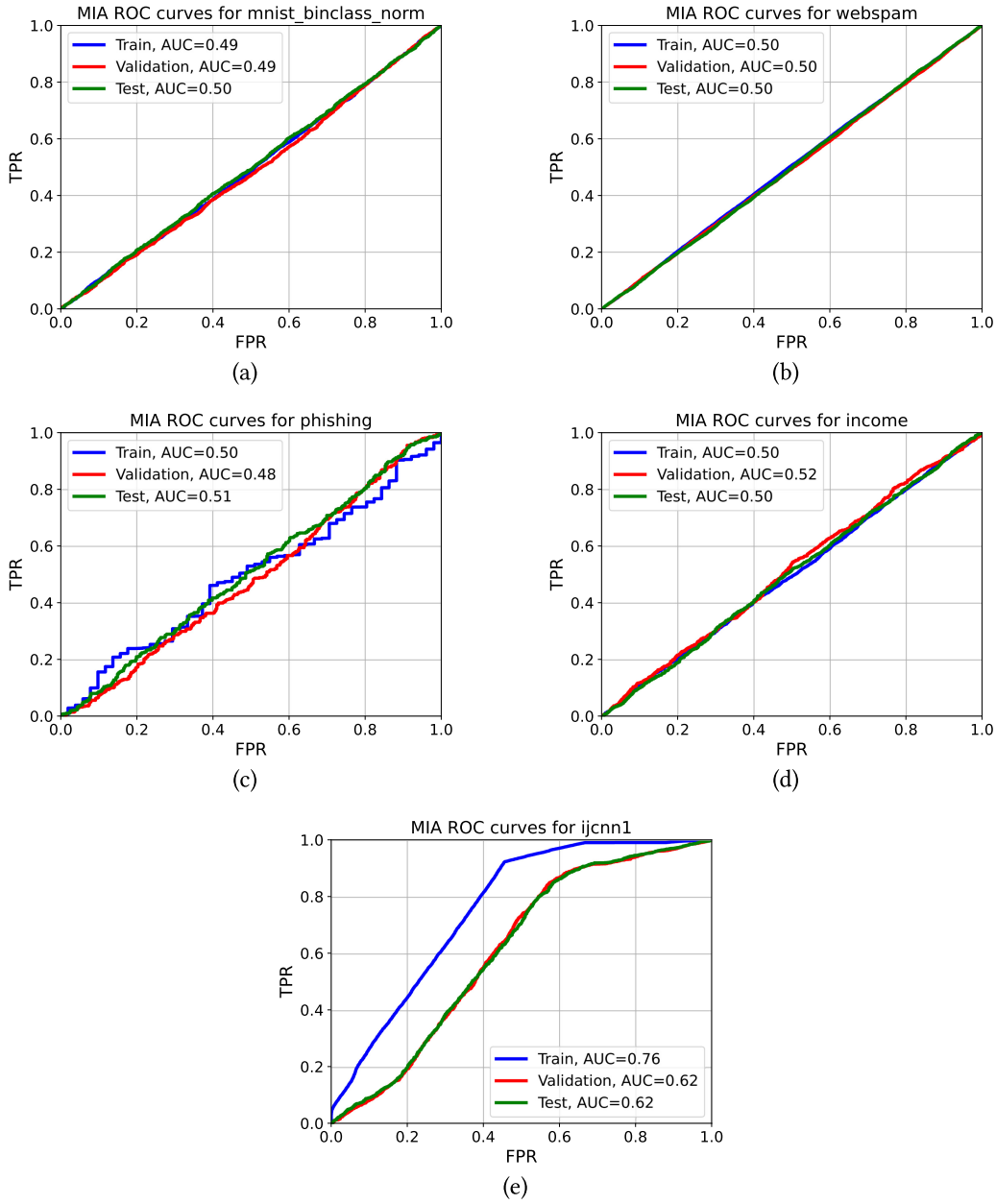


Fig. 10. Performance (ROC and AUC) of the MIA "omniscient" attacker for every dataset.

to avoid local divergence of the models in the workers. The BDSVM approach has proven to be completely resistant to non-ID data and converged at the times indicated in Table 4. We have run the MLP-FedAvg experiments using a maximum execution time of at least twice that of BDSVM to overcome the aforementioned convergence issues. Even with this additional training time, the performance of MLP-FedAvg was suboptimal in most cases, as can be seen in Table 3.

Table 3. Performance (AUC on the Test Set) of MLP-FedAvg vs. BDSVM in the Musketeer Platform Experiments

	MLP-FedAvg				BDSVM			
	4w	8w	16w	32w	4w	8w	16w	32w
Mnist(even/odd)	0.9365	0.9418	0.9451	0.9403	0.9969	0.9965	0.9968	0.9972
ijcnn1	0.8988	0.8932	0.8944	0.8601	0.9724	0.9876	0.9729	0.9710
phishing	0.9830	0.9803	0.9794	0.9790	0.9962	0.9964	0.9959	0.9947
income	0.9054	0.9063	0.9068	0.9011	0.9037	0.9056	0.9055	0.9036
webspam	0.8901	0.8615	0.8730	0.8570	0.9932	0.9931	0.9930	0.9928

Table 4. Training Time in Minutes of MLP-FedAvg vs. BDSVM in the Musketeer Real Platform Experiments

	MLP-FedAvg				BDSVM			
	4w	8w	16w	32w	4w	8w	16w	32w
Mnist(even/odd)	18.8	36.8	98.1	127.6	8.7	16.9	46.5	60.4
ijcnn1	4.5	16.7	15.2	25.9	1.9	7.8	6.8	11.7
phishing	6.3	8.4	19.4	41.3	2.8	3.7	8.7	18.7
income	56	113.6	172.7	131.1	27.3	56	83.8	62.4
webspam	15.9	9.2	30.7	33.8	7.6	4	14.1	14.5

Regarding algorithmic complexity, it is true that MLP-FedAvg requires fewer operations per iteration than BDSVM, but it also requires many more iterations to converge, at least in the non-ID scenario benchmarked in this article. That is, in our experiments, FedAvg required an average of 440 iterations (communication rounds) to converge, while BDSVM converged in approximately 3.3 iterations (average value). Using fewer iterations of FedAvg produced very poorly performing models. Furthermore, increasing the number of local epochs in each worker was also not helpful, as the local models had a rapid tendency to diverge due to the non-ID nature of the data. If we combine both factors (number of iterations and complexity/time per iteration), we obtain the training times indicated in Table 4, where we can see that BDSVM requires approximately half the time that MLP-FedAvg does to train the models, so in the end, BDSVM is advantageous even when it has higher computational cost per iteration.

4 CONCLUSIONS AND FURTHER WORK

FL has become a very popular paradigm that aims to train complex ML models on widely distributed data while preserving the data privacy, since no raw data is directly exchanged between the participants during the training process. FL is potentially capable of training models much faster than other Distributed Gradient approaches, because in its most popular version (e.g., FedAvg) local models are adjusted on each participant using many gradient steps computed on the available data and the resulting local models are averaged at a later stage at the aggregator, finally achieving global convergence in fewer epochs (communication rounds). However, the performance and efficiency of this approach is severely hampered when data is non-ID across participants, a situation that is very likely to occur in real-world situations. Although some solutions have been proposed to partially mitigate this problem, the performance issue is still present.

Kernel methods such as SVMs have been very popular in the past for solving many real-world problems with very competitive performance. Unfortunately, these methods have not undergone a comparable evolution in FL environments, compared to other architectures that can be trained through gradient descent. There are two reasons that make it difficult to use SVMs in a FL

environment. First of all, their architecture is determined by a subset of training patterns (known as SVs), and thus data confidentiality is at risk if we share SVs during training or even when exposing the resulting model. Second, the standard training process, based mainly on solving a Quadratic Programming optimization in the dual space, is computationally intensive and, in a distributed setting, it involves exchanging a large amount of potentially sensitive information.

We propose here a compact implementation of the SVM that, to our knowledge, is the first attempt to apply this technology in a FL setting. We take advantage of a BDSVM implementation whose model architecture does not contain any training pattern, and we use an alternative resolution method based on IRWLS.

We have benchmarked the proposed scheme in different scenarios using synthetic and real-world datasets, as well as virtual or real workers communicating through a cloud message exchange service. On the one hand, we observe that the proposed method is capable of providing maximum margin solutions, which are the key to the good performance of SVMs in many problems. Second, we show that the method is practically insensitive with respect to non-ID data partitions, since the functional that is minimized in the FL configuration is shown to be essentially equivalent to that for a centralized setup, unlike the FedAvg approach, which is very sensitive to non-ID scenarios, and ultimately produces less competitive solutions. We show that even with a large number of participants the performance is not affected and the transmission cost is competitive with respect to FedAvg, since the latter requires many more epochs to converge in the non-ID scenario (fewer local training steps can be applied due to local model divergence problems). Finally, we also show the resilience of the proposed scheme in situations where part of the information is lost during training (dropout, straggler participants), as well as its robustness against information leakage and membership inference attacks.

SOFTWARE AND DATA

The developed algorithms are the result of the EU-H2020 research project MUSKETEER, and all the developed libraries will be released as open source code at www.musketeer.eu, according to the project's Grant Agreement.

REFERENCES

[1] S. Bonura, D. Dalle Carbonare, R. Díaz-Morales, M. Fernández-Díaz, L. Morabito, L. Muñoz-González, C. Napione, A. Navia-Vázquez, and M. Purcell. 2020. Privacy preserving technologies for trusted data spaces. *Technologies and Applications for Big Data Value*. Big Data Value Association (BDVA).

[2] T. Brisimi, R. Chen, T. Mela, A. Olshevsky, I. Paschalidis, and W. Shi. 2018. Federated learning of predictive models from federated electronic health records. *International Journal of Medical Informatics* 112 (2018), 59–67.

[3] C. Caragea, D. Caragea, and V. Honavar. 2005. Learning support vector machine classifiers from distributed data sources. In *Proc. 20th National Conference on Artificial Intelligence*.

[4] R. Carlsson. 2020. *Privacy-Preserved Federated Learning: A Survey of Applicable Machine Learning Algorithms in a Federated Environment*. Ph.D. Dissertation, Uppsala University.

[5] R. Collobert, S. Bengio, and Y. Bengio. 2002. A parallel mixture of SVMs for very large scale problems. *Neural Computation* 14, 5 (2002), 1105–1114.

[6] Council of European Union. 2014. Council regulation (EU) no. 269/2014. http://eur-lex.europa.eu/legal-content/EN/TXT/?qid=1416170084502&uri=CELEX: 32014R0269.

[7] R. Cramer, I. B. Damgård, and J. B. Nielsen. 2015. *Secure Multiparty Computation*. Cambridge University Press.

[8] R. Díaz-Morales and A. Navia-Vázquez. 2018. Distributed nonlinear semiparametric support vector machine for big data applications on spark frameworks. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 50, 11 (2018), 4664–4675.

[9] T. N. Do and F. Poulet. 2006. Classifying one billion data with a new distributed SVM algorithm. In *Proc. International Conference on Research, Innovation and Vision for the Future*. 59–66.

[10] D. Dua and C. Graff. 2019. MUCI Machine Learning Repository, University of California, School of Information and Computer Science, Irvine, CA.

[11] C. Dwork. 2011. Differential privacy. *Encyclopedia of Cryptography and Security*. Springer, Boston, MA.
[12] A. Artés-Rodríguez, F. Pérez-Cruz, and C. Bousoño-Calzón. 2005. Convergence of the IRWLS procedure to the support vector machine solution. *Neural Computation* 17 (2005), 7–18.
[13] K. Flouri, B. Beferull-Lozano, and P. Tsakalides. 2006. Distributed consensus algorithms for SVM training in wireless sensor networks. In *Proc. EUSIPCO'08*.
[14] K. Flouri, B. Beferull-Lozano, and P. Tsakalides. 2006. Training a support-vector machine-based classifier in distributed sensor networks. In *Proc. of the 14th European Signal Processing Conference*.
[15] P. A. Forero, A. Cano, and G. B. Giannakis. 2010. Consensus-based distributed linear support vector machines. In *Proc. of the International Conference on Information Processing in Sensor Network*.
[16] P. A. Forero, A. Cano, and G. B. Giannakis. 2010. Consensus-based distributed support vector machines. *Journal of Machine Learning Research* 11 (2010), 1663–1707.
[17] H. P. Graf, E. Cosatto, L. Bottou, I. Dourdanovic, and V. Vapnik. 2005. Parallel support vector machines: The cascade SVM. *Advances in Neural Information Processing Systems*, Vol. 17. 521–528.
[18] M. Hardt, B. Recht, and Y. Singer. 2016. Train faster, generalize better: Stability of stochastic gradient descent. In *Proc. International Conference on Machine Learning (ICML'16)*.
[19] V. Hartmann, K. Modi, J. M. Pujol, and R. West. 2020. Privacy preserving classification with secret vector machines. In *Proc. of the 29th ACM International Conference on Information & Knowledge Management*. 475–484.
[20] T. Hazan, A. Man, and A. Shashua. 2008. A parallel decomposition solver for SVM: Distributed dual ascend using Fenchel Duality. In *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR'08)*. 1–8.
[21] K. Hu, Y. Li, M. Xia, J. Wu, M. Lu, S. Zhang, and L. Weng. 2021. Federated learning: A distributed shared machine learning method. *Complexity* (2021), 1–20.
[22] G. B. Huang. 2015. What are extreme learning machines? Filling the gap between Frank Rosenblatt's dream and John von Neumann's puzzle. *Cognitive Computation* 7, 3 (2015), 263–278. https://doi.org/10.1007/s12559-015-9333-0
[23] G. B. Huang, Q. Y. Zhu, and C. Siew. 2006. Extreme learning machine: Theory and applications. *Neurocomputing* 70, 1 (Dec. 2006), 489–501. https://doi.org/10.1016/j.neucom.2005.12.126
[24] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, et al. 2019. Advances and open problems in federated learning. arXiv:1912.04977. https://arxiv.org/pdf/1912.04977.pdf.
[25] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh. 2020. SCAFFOLD: Stochastic controlled averaging for federated learning. In *International Conference on Machine Learning*. PMLR, 5132–5143.
[26] J. Konečný, H. B. McMahan, D. Ramage, and P. Richtárik. 2016. Federated optimization: Distributed machine learning for on-device intelligence. arXiv:1610.02527 [cs.LG]. https://arxiv.org/pdf/1610.02527.pdf.
[27] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. 1998. Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86 (Dec. 1998), 2278–2324. https://doi.org/10.1109/5.726791
[28] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith. 2018. Federated optimization in heterogeneous networks. arXiv:1812.06127. https://arxiv.org/abs/1812.06127.
[29] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou. 2021. FedBN: Federated learning on non-IID features via local batch normalization. arXiv:2102.07623. https://arxiv.org/abs/2102.07623.
[30] O. L. Mangasarian, E. W. Wild, and G. M. Fung. 2008. Privacy-preserving classification of vertically partitioned data via random kernels. *ACM Transactions on Knowledge Discovery from Data* 12 (2008), 1–12.
[31] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera-Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. arXiv:1602.05629v2. https://arxiv.org/pdf/1602.05629.pdf.
[32] R. Mohammad, F. Thabtah, and T. Mccluskey. 2012. An assessment of features related to phishing websites using an automated technique. In *Proc. International Conference for Internet Technology and Secured Transactions (ICITST'12)*. 492–497.
[33] A. Navia-Vázquez, D. Gutiérrez-González, E. Parrado-Hernández, and J. J. Navarro-Abellán. 2006. Distributed support vector machines. *IEEE Transactions on Neural Networks,* 17 (Aug. 2006), 1091–1097. https://doi.org/10.1109/TNN.2006.875968
[34] A. Navia-Vázquez, F. Pérez-Cruz, A. Artés Rodríguez, and A. Figueiras-Vidal. 2001. Weighted least squares training of support vector classifiers leading to compact and adaptive schemes. *IEEE Transactions on Neural Networks* 12 (Feb. 2001), 1047–1059. https://doi.org/10.1109/72.950134
[35] E. Parrado-Hernández, I. Mora-Jiménez, J. Arenas-García, A. R. Figueiras-Vidal, and A. Navia-Vázquez. 2003. Growing support vector classifiers with controlled complexity. *Pattern Recognition* 36, 7 (2003), 1479–1488.
[36] D. Pechyony, L. Shen, and R. Jones. 2011. Solving large scale linear SVM with distributed block minimization. In *Proc. NIPS 2011 Workshop on Big Learning: Algorithms, Systems and Tools for Learning at Scale*.
[37] D. Prokhorov. 2001. IJCNN 2001 neural network competition, Ford Research Laboratory.

[38] F. Pérez-Cruz. 2000. *MÃąquina de Vectores Soporte Adaptativa y Compacta*. Ph.D. thesis, Universidad PolitÃĺcnica de Madrid (in Spanish).
[39] F. Pérez-Cruz, P. L. Alarcón-Diana, A. Navia-Vázquez, and A. Artés-Rodríguez. 2000. Fast training of support vector machines. In *Advances in Neural Information Processing Systems (NIPS'00)*, Vol. 13. 734–740.
[40] Y. Rahulamathavn, R. C.-W. Phan, S. Veluru, and K. Cumanan. 2014. Privacy-preserving multi-class support vector machine for outsourcing the data classification in cloud. *IEEE Transactions on Dependable and Secure Computing* 11, 5 (2014), 467–479.
[41] F. Rosenblatt. 1958. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review* 65, 6 (1958), 386–408.
[42] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes. 2019. ML-leaks: Model and data independent membership inference attacks and defenses on machine learning models. In *Proc. of the 26th Annual Network and Distributed System Security Symposium (NDSS'19)*.
[43] A. Salem, Y. Zhang, M. Humbert, P. Berrang, and M. Fritz, and M. Backes. 2019. ML-leaks: Model and data independent membership inference attacks and defenses on machine learning models. In *Proc. of the 26th Annual Network and Distributed System Security Symposium (NDSS'19)*.
[44] B. Scholkopf, C. Burges, and A. Smola. (Eds.) 1998. Advances in Kernel Methods - Support Vector Learning. The MIT Press.
[45] M. Shen, X. Tang, L. Zhu, X. Du, and M. Guizani. 2019. Privacy-preserving support vector machine training over blockchain-based encrypted IoT data in smart cities. *IEEE Internet of Things Journal* 6, 5 (2019), 7702–7712.
[46] N. Shlomo. 2018. Disclosure limitation: New directions and challenges. *Journal of Privacy and Confidentiality* 8 (2018).
[47] R. Shokri, M. Stronati, C. Song, and V. Shmatikov. 2017. Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy (SP'17)*. 3–18.
[48] D. Singh and C. K. Mohan. 2018. Projection-SVM: Distributed kernel support vector machine for big data using subspace partitioning. In *IEEE International Conference on Big Data (Big Data'18)*. 74–83.
[49] M. Sinn, R. Díaz, L. Muñoz, D. Dalle Carbonare, A. Navia, L. Nagel, S. Rosello, G. Fecondo, L. Morabitto, J. Correia, and P. Papachristou. 2018. MUSKETEER: Machine learning to augment shared knowledge in federated privacy-preserving scenarios. European Union's Horizon 2020 Research and Innovation Programme under Grant Agreement No. 824988. http://www.musketeer.eu.
[50] V. Smith, C.-K. Chiang, M. Sanjabi, and A. Talwalkar. 2017. Federated multi-task learning. arXiv:1705.10467. https://arxiv.org/abs/1705.10467.
[51] N. A. Syed, S. Huan, L. Kah, and K. Sung. 1999. Incremental learning with support vector machines. In *Proc. of the 5th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
[52] T. Timan and Mann Z. A. (Eds.). 2019. *Data Protection in the Era of Artificial Intelligence. Trends, Existing Solutions and Recommendations for Privacy-Preserving Technologies*. Big Data Value Association (BDVA) position paper.
[53] J. Vaidya, H. Yu, and X. Jiang. 2008. Privacy-preserving SVM classification. *Knowledge and Information Systems* 14, 2 (2008), 161–178.
[54] S. Webb, J. Caverlee, and C. Pu. 2006. Introducing the webb spam corpus: Using email spam to identify web spam automatically. In *Proc. of the 3rd Conference on Email and Anti-Spam (CEAS'06)*.
[55] T. Xu, T. S. Brisimi, T. Wang, W. Dai, and I. C. Paschalidis. 2016. A joint sparse clustering and classification approach with applications to hospitalization predictions. In *Proc. of the IEEE 55th Conference on Control (CDC'16)*. 4566–4571.
[56] K. Yang, T. Jiang, Y. Shi, and Z. Ding. 2020. Federated learning via over-the-air computation. *IEEE Transactions on Wireless Communications* 19, 3 (2020), 2022–2035.
[57] K. Yang, Y. Shi, Y. Zhou, Z. Yang, L. Fu, and W. Chen. 2020. Federated machine learning for intelligent IoT via reconfigurable intelligent surface. *IEEE Network* 34 (2020), 16–22.
[58] Q. Yang, Y. Liu, T. Chen, and Y. Tong. 2019. Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology* 10 (2019), 1–19.
[59] Z. Yang and W. U. Bajwa. 2016. A resilient distributed support vector machine. In *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP'16)*. 2444–2448.
[60] H. Yu, X. Jiang, and J. Vaidya. 2006. Privacy-preserving SVM using nonlinear kernels on horizontally partitioned data. In *Proc. 2006 ACM Symposium on Applied computing (SAC'06)*. 603–610.
[61] H. Yunhong, F. Liang, and H. Guoping. 2009. Privacy-preserving SVM classification on vertically partitioned data without secure multi-party computation. In *Proc. of the 5th International Conference on Natural Computation (ICNC'09)*, Vol. 1. 543–546.
[62] G. Zanghirati and L. Zanni. 2003. A parallel solver for large quadratic programs in training support vector machines. *Parallel Computing* 29, 4 (2003), 535–551.
[63] V. Zantedeschi, A. Bellet, and M. Tommasi. 2019. Communication-efficient and decentralized multi-task boosting while learning the collaboration graph. arXiv:1901.08460. https://aps.arxiv.org/abs/1901.08460v1.

[64] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra. 2018. Federated learning with non-iid data. arXiv:1806.00582. https://arxiv.org/abs/1806.00582.

[65] L. Zhu, Z. Liu, and S. Han. 2019. Deep leakage from gradients. In *Proc. of the Advances in Neural Information Processing Systems 32 (NeurIPS'19).*

Received March 2021; revised April 2022; accepted May 2022